

FINAL PROJECT REPORT

E-Commerce Online Shop System

Capstone Project Group One

Team: Weijie, Shukla, and Jade

Project start: 21 July 2026

Report revision: 29 July 2026

Solution: React/Vite client, Express API, Vercel deployment, and Neon PostgreSQL

A consolidated record of scope, design, implementation, testing, deployment, and submission readiness.

Executive Summary

Capstone Project Group One developed a full-stack e-commerce application that supports the complete shopping journey from product discovery to order confirmation. Visitors can browse as guests, while registered members can authenticate, access protected features, and receive the documented membership benefit. The solution integrates a React/Vite client, an Express REST API, and a Neon-hosted PostgreSQL database, with the deployable applications hosted on Vercel.

The minimum viable product (MVP) prioritizes a reliable end-to-end transaction over advanced retail features. Products and categories are retrieved from the database; shoppers can search, filter, inspect products, and manage a cart; checkout supports both guest and member orders; and the server recalculates prices and discounts before persisting orders and order items. Online payment processing is intentionally excluded.

Project outcome: The project is considered demonstration-ready when the deployed client communicates with the deployed API, authentication and shopping flows work reliably, orders persist in PostgreSQL, and supporting documentation and test evidence are complete.

Document Purpose

This report replaces the original working notebook with a structured project record. It consolidates requirements, user stories, system design, interface plans, database and API decisions, implementation priorities, test expectations, deployment links, and the final submission checklist. Repeated notes have been merged, while the original wireframes, ER diagram, and seed-data reference have been retained.

Contents

- Project overview and scope
- User journeys, requirements, and user stories
- Technology, architecture, routes, and interface design
- Database model, API contract, and business rules
- Implementation plan, testing approach, and deployment
- Completion checklist and SQL seed-data appendix

1. Project Overview

1.1 Purpose and Objectives

The project demonstrates the design and delivery of a modern full-stack web application. Its purpose is to provide a clear, responsive online shop in which users can discover products, manage a cart, submit an order, and retrieve confirmation or tracking information.

- Deliver a usable public storefront with a clear shop introduction.
- Support guest shopping without forcing account creation.
- Provide secure member registration, login, logout, and protected access.
- Retrieve product and category data through a REST API backed by PostgreSQL.
- Persist completed orders and their line items in the database.
- Deploy the client, administration interface, API, and database to cloud services.
- Produce sufficient documentation and evidence for assessment and demonstration.

1.2 Current Project Record

Date	Progress recorded
21 Jul 2026	Scope confirmed; user stories, wireframes, ER diagram, API endpoints, repository, and the initial React/Vite scaffold were started or completed.
22 Jul 2026	Neon database created and initial database tables added.
25 Jul 2026	Order and order-tracking flows improved; five product images added to the backend; login and registration flows corrected.
25 Jul 2026 discussion	Next priorities: product administration, a stronger landing page, and a language selector.
28 Jul 2026	Create Administrator React
29 Jul 2026	Login page for Administrator React Add Theme & Language for Client page

1.3 Definition of Done

- Members can register, log in, refresh an authenticated session, access the dashboard, and log out.
- Guests can browse, manage a cart, and complete checkout without registration.
- Products and categories are retrieved from PostgreSQL and displayed correctly.
- Search and category filtering return clear results and empty states.
- Checkout persists orders and order items using server-calculated prices.
- The member discount is calculated by the backend when enabled.

- Order confirmation displays information retrieved from the persisted order.
- The primary interface supports two languages and remembers the selected language.
- The deployed frontend communicates successfully with the deployed backend.
- Required documentation, screenshots, test evidence, and presentation materials are complete.

2. Scope

2.1 Required MVP Features

Area	Required capability
Authentication and access	Member registration, login, logout, JWT session management, password hashing, validation, guest access, and a protected member dashboard.
Shop experience	Homepage introduction, category and product listing, SKU display, product details, search, product imagery, loading states, empty states, and responsive layouts.
Cart	Add, update, and remove items; maintain quantities; calculate summaries; and prevent checkout with an empty cart.
Checkout and orders	Guest and member checkout, order confirmation, database persistence for orders and line items, and backend price recalculation. No payment gateway is required.
Technical delivery	React/Vite, React Router, Express, REST APIs, PostgreSQL, Git/GitHub, frontend-backend integration, Vercel deployment, and Neon hosting.
Documentation	API documentation, ER diagram, wireframes, test cases, bug log, deployment guide, README, final report, and project presentation.

2.2 Optional Enhancements

- Product sorting, pagination, featured-product sections, and richer category browsing.
- Bilingual product names and descriptions; the MVP may translate only interface content.
- Member order history, pre-filled checkout details, and a persistent member cart.
- Product administration for adding, editing, and deactivating products.
- Loading skeletons, toast notifications, image carousels, animations, and dark mode.
- Advanced responsive refinements after the core flow is stable.

Scope rule: Optional features must not delay or destabilize the complete product-to-order flow. If time is limited, product administration may be demonstrated through API requests instead of a full admin interface.

2.3 Out of Scope

- Password recovery, email verification, social login, and multi-factor authentication.
- Wishlists, reviews, ratings, recommendations, coupons, and promotional engines.
- Credit-card or online payment processing, including Stripe, PayPal, Razorpay, or UPI.
- Shipment notifications, invoice generation, returns, refunds, and stock alerts.
- Supplier, warehouse, analytics, sales-reporting, and complex inventory functions.
- Profile images, address books, and multiple saved shipping addresses.
- Complex roles, OAuth, audit logging, Redis, CDN configuration, and load balancing.

3. User Experience and Functional Requirements

3.1 Primary Shopping Journey

1. Visit the online shop.
2. Continue as a guest or register and log in as a member.
3. Read the shop introduction and browse product categories.
4. Search or filter the catalogue.
5. Open a product to review its details.
6. Add products to the cart and update or remove items as required.
7. Proceed to checkout.
8. Submit customer and delivery information.
9. Save the order and display the confirmation or tracking view.

Terminology: Use “Continue as guest” rather than “Guest Login.” A guest does not authenticate; the shopper simply continues without an account.

3.2 Navigation and Routes

The public navigation should remain consistent across the site: Logo | Home | Products | Categories | Language | Cart | Login/Account.

Access	Route	Purpose
Public	/	Shop introduction, featured categories, and language selector
Public	/products	All active products, search, sorting, and category filters
Public	/products/:id	Product details and add-to-cart action

Public	/login	Member login
Public	/register	Member registration
Public	/cart	Cart contents and totals
Public	/checkout	Guest or member checkout form
Public	/order-confirmation/:id	Persisted order confirmation or tracking view
Member	/dashboard	Protected welcome page and membership information
Member	/orders	Optional member order history
Admin	/admin/products	Optional product-management listing
Admin	/admin/products/new	Optional product creation
Admin	/admin/products/:id/edit	Optional product editing

3.3 Language Selection

The MVP should support English and Chinese for the main interface. Navigation, buttons, form labels, validation messages, cart and checkout text, and the shop introduction should be translated. The selected language should be stored in localStorage and restored after a refresh.

```
src/locales/
├── en.json
└── zh.json
```

Product names and descriptions may remain in the default language during the first implementation. If translated catalogue content is required, use dedicated fields such as name_en, name_zh, description_en, and description_zh.

3.4 Membership Discount

The intended MVP rule is deliberately simple and must be calculated by the backend:

```
Guest: discount = 0
Logged-in member: discount = subtotal × 10%
Total = subtotal – discount
```

Calculation	Amount
Subtotal	\$100.00
Member discount (10%)	-\$10.00
Final total	\$90.00

The percentage should be stored in one backend configuration value. Coupon codes, multiple membership tiers, expiry rules, and promotional campaigns are excluded from the MVP unless the project scope is formally expanded.

4. User Stories and Acceptance Criteria

The following stories define the assessable behavior of the MVP. “Must have” stories are required for the demonstration; “Should have” and “Could have” stories may be completed after the core journey is stable.

US-01 — Browse as a guest

Priority: Must have

As a visitor, I want to browse and shop without creating an account.

- Unauthenticated visitors can browse products and categories.
- Guests can add items to a cart and proceed to checkout.
- The application does not create a user account automatically.

US-02 — Register as a member

Priority: Must have

As a visitor, I want to register so that I can access member benefits.

- The form collects name, email, password, and password confirmation.
- Valid submissions create a member; duplicate emails and invalid values return clear errors.
- Passwords are hashed, and password data is never returned by the API.

US-03 — Log in as a member

Priority: Must have

As a registered member, I want to log in securely and access member features.

- Valid credentials establish a JWT-based session and redirect the user.
- Invalid credentials return a general error without revealing sensitive details.
- The session survives a refresh, and the navigation displays the member state.

US-04 — Log out

Priority: Must have

As a member, I want to end my session so that my account remains secure.

- Logout clears the authentication session and blocks protected routes.
- The navigation returns to its logged-out state.
- Cart contents remain unless removal is an intentional business rule.

US-05 — View the homepage

Priority: Must have

As a shopper, I want to understand the shop and begin browsing quickly.

- The page presents the shop name, introduction, navigation, products, and categories.
- Login, language, and cart controls are accessible.

- The page remains usable on desktop and mobile screens.

US-06 — Select a language

Priority: Must have

As a shopper, I want to use the interface in my preferred language.

- At least two languages are available for interface content.
- The selected language updates navigation, controls, labels, and messages.
- The selection persists in browser storage, with fallback to the default language.

US-07 — Browse products

Priority: Must have

As a shopper, I want to review available products before purchasing.

- Active products are retrieved from PostgreSQL through the backend.
- Each card shows an image, SKU, name, price, and category.
- Loading, empty, and error states are clear and helpful.

US-08 — Browse by category

Priority: Must have

As a shopper, I want to filter products by category.

- Categories are retrieved from the database.
- Selecting a category shows only matching active products and identifies the selection.
- All Products restores the complete catalogue; empty categories show an appropriate message.

US-09 — Search for products

Priority: Must have

As a shopper, I want to search by product name or SKU.

- A search field is available on the product page.
- Search is case-insensitive and can be combined with category filtering.
- No-match and request-failure states are displayed clearly.

US-10 — View product details

Priority: Must have

As a shopper, I want enough detail to make a purchase decision.

- The detail page shows image, SKU, name, category, price, description, and availability.
- Invalid or inactive product identifiers return a suitable message.
- The shopper can choose a valid quantity and add the product to the cart.

US-11 — Add a product to the cart

Priority: Must have

As a shopper, I want to add products to my cart.

- The selected product and quantity appear in the cart.
- Adding an existing product updates its quantity rather than creating a confusing duplicate.
- The cart indicator and calculated totals update immediately.

US-12 — Manage the cart

Priority: Must have

As a shopper, I want to review and change my cart before checkout.

- Each line displays product details, unit price, quantity, and line total.
- The shopper can change quantities or remove items.
- Invalid quantities are prevented, and an empty cart cannot proceed to checkout.

US-13 — Receive a member discount

Priority: Should have — scope extension

As a logged-in member, I want the documented discount applied consistently.

- The backend determines membership status and calculates the discount.
- Subtotal, discount, and final total are displayed clearly.
- Guests do not receive the member discount, and the stored order records the final calculation.

US-14 — Complete guest checkout

Priority: Must have

As a guest, I want to place an order without registering.

- The form collects name, email, phone, and shipping address.
- The backend validates the cart, retrieves current prices, and saves the order and items.
- A secure confirmation identifier provides access to the guest order.

US-15 — Complete member checkout

Priority: Must have

As a member, I want my authenticated order associated with my account.

- The checkout recognizes the authenticated user and may pre-fill known details.
- The backend calculates any approved member discount and links the order to the member.
- Successful submission clears the cart and opens confirmation.

US-16 — View order confirmation

Priority: Must have

As a shopper, I want confirmation that my order was saved.

- The page displays the order number, items, customer details, subtotal, discount, total, status, and date.
- Information is retrieved from the persisted order, not only from local state.
- Unauthorized users cannot view an order that does not belong to them.

US-17 — View the member dashboard

Priority: Must have

As a member, I want a protected account area.

- Unauthenticated users are redirected to login.
- The page shows a personalized greeting and relevant membership information.
- Authenticated users can reach order history when that feature is enabled.

US-18 — View previous orders

Priority: Could have

As a member, I want to review my previous purchases.

- Only authenticated members can access their own order history.
- Each record shows the order number, date, status, and total.
- Opening a record displays its items and complete total calculation.

US-19 — Handle errors clearly

Priority: Must have

As a user, I want understandable feedback when an action fails.

- Forms show field-level validation and requests cannot be submitted repeatedly while processing.
- Login and checkout failures always leave the interface in a usable state.
- Technical details, database errors, and password information are not exposed.

US-20 — Support common screen sizes

Priority: Must have

As a shopper, I want to use the application on desktop and mobile devices.

- Navigation, cards, forms, cart, and checkout do not overlap or require horizontal scrolling.
- Fields have visible labels, and controls are keyboard-operable.
- Product images include meaningful alternative text.

5. Solution Architecture and Technology

5.1 Technology Stack

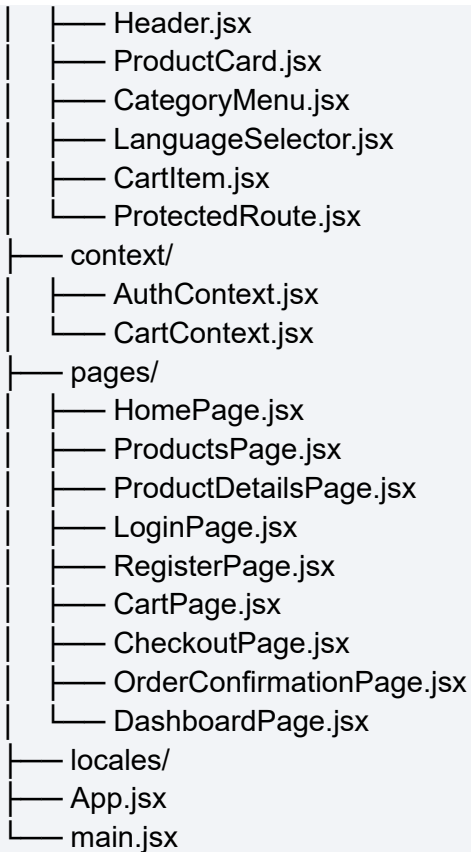
Area	Selected technology	Purpose
Client	Vite, React, JavaScript	Responsive storefront and account experience
Routing	React Router	Public, protected, and optional admin routes
Styling	Bootstrap or plain CSS	Responsive layout and consistent components
Translation	react-i18next	Two-language interface with persistent preference
Forms	React Hook Form or controlled forms	Input collection and validation
API	Node.js and Express	REST endpoints and business rules
Database	PostgreSQL on Neon	Products, users, orders, and optional carts
Authentication	JWT and bcryptjs	Session management and password hashing
Database access	pg	Parameterized PostgreSQL queries
Validation	Zod or server-side validation	Request validation and safe error responses
Deployment	Vercel and Neon	Hosted client, API, admin app, and database
Version control	Git and GitHub	Team collaboration and source history

5.2 Logical Architecture

1. The React client requests products, categories, authentication, and order services from the Express API.
2. The API validates requests, verifies JWTs where required, and executes business rules.
3. Parameterized queries read and write data in the Neon PostgreSQL database.
4. During checkout, the API retrieves current products and prices rather than trusting totals supplied by the browser.
5. The API stores the order header and item snapshots, then returns persisted confirmation data.

5.3 Suggested Frontend Structure

```
src/  
├── api/  
│   ├── authApi.js  
│   ├── productApi.js  
│   └── orderApi.js  
└── components/
```



AuthContext should maintain the current member and authentication state. CartContext should own cart actions, item counts, and displayed totals.

6. Interface Design

6.1 Homepage Wireframe

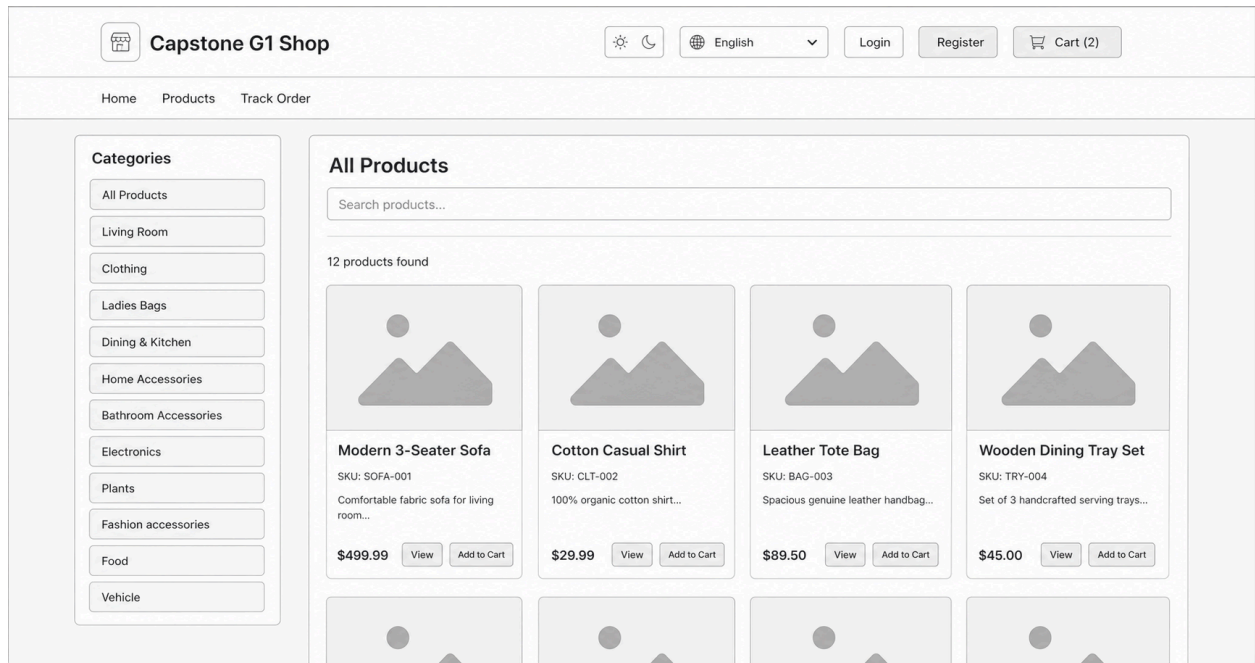


Figure 1. Desktop homepage wireframe showing navigation, language selection, category discovery, featured products, and member/guest entry.

6.2 Supporting Page Wireframes

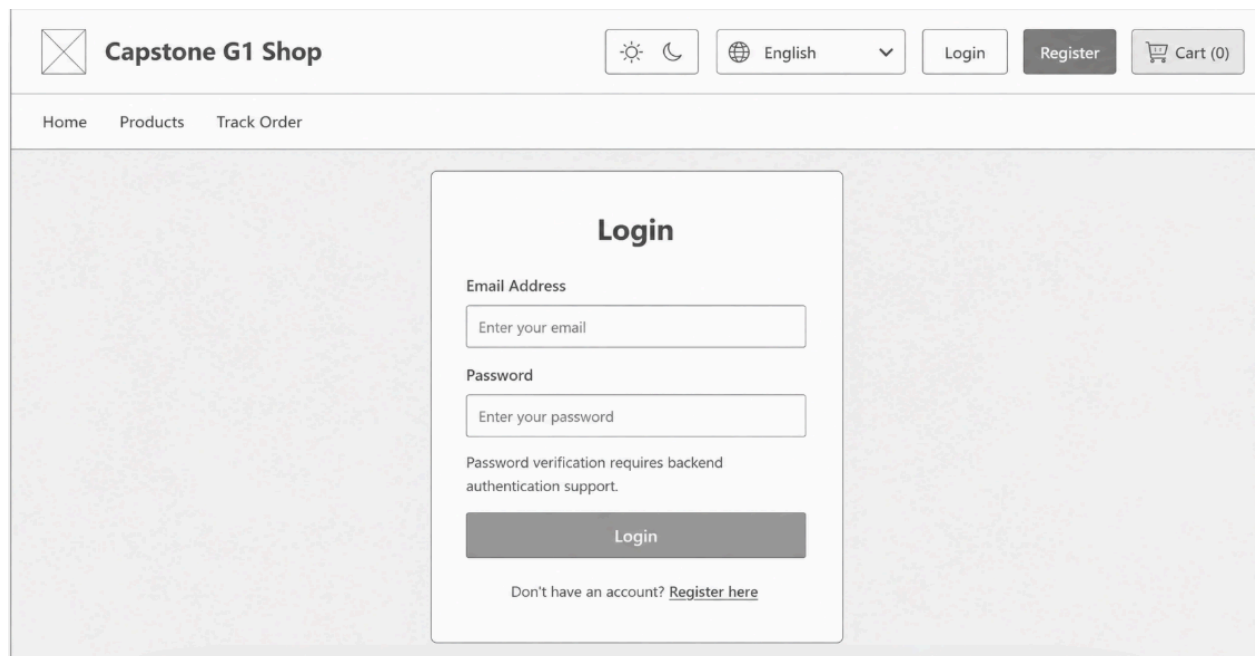


Figure 2. Member login page.

**Capstone G1 Shop**



English

Login

Register

Cart (1)

[Home](#) [Products](#) [Track Order](#)

Shopping Cart

Product	Quantity	Subtotal	Action
 Modern 3-Seater Sofa SKU: SOFA-001 \$499.99	- 1 +	\$499.99	Remove

Order Summary

Subtotal:

\$499.99

Shipping:

\$0.00

Tax:

\$0.00

Total:

\$499.99

Proceed to Checkout

Continue Shopping

Figure 3. Shopping cart page.

**Capstone G1 Shop**



English

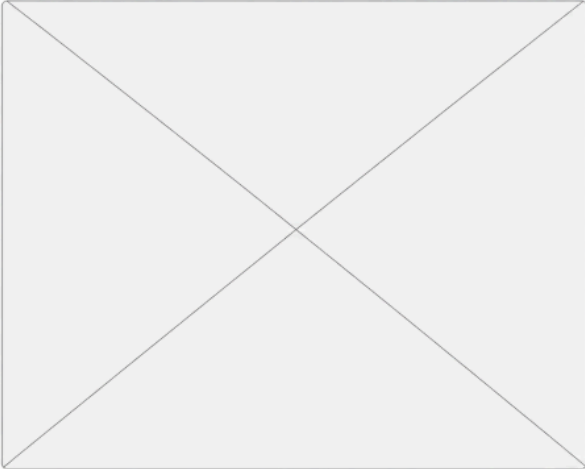
Login

Register

Cart (1)

[Home](#) [Products](#) [Track Order](#)

← Back to Products



X1
SKU: ELE-001
★★★★☆ (4.5/5) (128 reviews)
\$900.00

✓ In Stock

Description
X1 Gaming console

Quantity:

-

1

+

Add to Cart

View Cart

Category: Uncategorized

Added: 7/29/2026

Figure 4. Product details page.

Capstone G1 Shop

Home Products Track Order

Track Guest Order

Enter the order number shown on your checkout confirmation.

Order Number

ORD-1785331094333-95f6dae4

View Order

ORDER NUMBER	STATUS	TOTAL
#ORD-1785331094333-95f6dae4	PENDING	\$499.99

Figure 5. Order Tracking page

6.3 Interface Principles

- Keep the primary navigation, language selector, cart indicator, and account state visible and consistent.
- Make product SKU, name, image, price, category, and availability easy to scan.
- Use explicit labels for all form fields and provide messages beside invalid inputs.
- Provide loading, empty, success, and failure states for every data-driven page.
- Use responsive layouts that do not overlap or create horizontal scrolling.
- Ensure keyboard operation and meaningful alternative text for product imagery.

7. Database Design

7.1 Entity Relationship Diagram

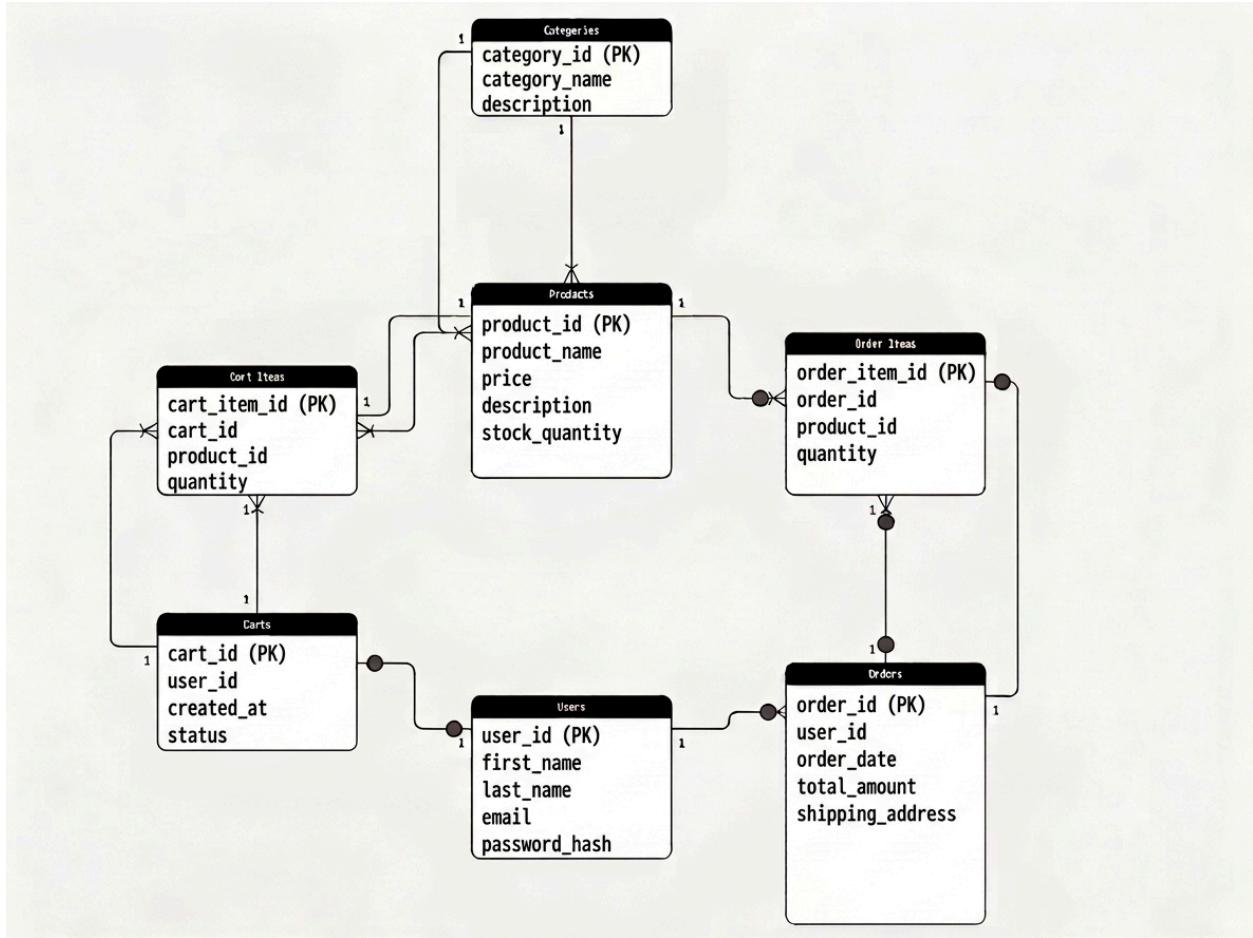


Figure 6. PostgreSQL entity relationship model for member and guest shopping.

7.2 Core Entities

Entity	Key fields and purpose
users	id, name, email, password_hash, membership_type or discount_rate, timestamps
categories	id, name, slug, timestamps
products	id, category_id, sku, name/title, description, price, image_url, is_active, timestamps
orders	id, nullable user_id, order number, confirmation token, customer details, shipping address, subtotal, discount, total, status, created_at
order_items	id, order_id, product_id, SKU, product name/title, unit price, quantity, line total
carts (optional)	id, nullable user_id, guest_session_id or session token, timestamps
cart_items (optional)	id, cart_id, product_id, quantity, timestamps

7.3 Design Decisions

- orders.user_id is nullable so guests can place orders.
- Guest contact and delivery details are stored directly on the order.

- A secure confirmation token can identify a guest order without requiring login.
- Order items store snapshots of SKU, product name, and purchase price so historical orders remain accurate after catalogue changes.
- Calculated discount values are stored on each order.
- Carts can support both members and guests; however, carts and cart items may be removed from the database if the MVP stores cart state only in localStorage.

7.4 Data Integrity and Security

- Use unique constraints for email, category slug, product SKU, order number, and confirmation token where applicable.
- Use foreign keys for category/product, order/order-item, and optional cart/cart-item relationships.
- Use parameterized SQL through pg to prevent injection.
- Never expose password hashes in API responses.
- Wrap order creation and order-item insertion in a transaction so partial orders are not saved.
- Validate prices, quantities, customer data, and ownership on the server.

8. REST API Contract

Development base URL: <http://localhost:3000/api/v1>

Production base URL: use the deployed Vercel API origin. Request and response bodies use JSON.

8.1 Core Endpoints

Method	Endpoint	Purpose
POST	/api/auth/register	Register a new member
POST	/api/auth/login	Authenticate a member and establish a session
GET	/api/auth/me	Retrieve the authenticated member
POST	/api/auth/logout	End the current session
GET	/api/categories	List product categories
GET	/api/products	List active products; accept search and category queries
GET	/api/products/:id	Retrieve one product
POST	/api/orders	Validate the cart and create an order
GET	/api/orders/:id	Retrieve an authorized order or guest confirmation
GET	/api/orders/my-orders	Optional member order history
POST	/api/products	Optional product administration
PUT	/api/products/:id	Optional product update
DELETE	/api/products/:id	Optional product deactivation or deletion

8.2 Example Registration Exchange

```
POST /api/v1/auth/register
```

```
{
  "name": "Alex Tan",
  "email": "alex@example.com",
  "password": "Password123!"
}
```

```
201 Created
```

```
{
  "message": "Registration successful",
  "user": { "id": 12, "name": "Alex Tan", "email": "alex@example.com" }
}
```

8.3 Product Query Examples

```
GET /api/products?search=shirt
```

```
GET /api/products?category=clothing
```

```
GET /api/products?search=shirt&category=clothing
```

8.4 Response Status Standards

Code	Meaning
200	Request completed successfully
201	Resource created successfully
400	Invalid request or validation failure
401	Authentication required or token invalid
403	Authenticated user lacks permission
404	Resource not found
409	Conflict, such as duplicate email or SKU
500	Unexpected server error

8.5 Checkout Trust Boundary

Security requirement: The API must never trust totals submitted by React. It must retrieve current products and prices, validate quantities and availability, calculate the subtotal and approved discount, then save the order and line items within a database transaction.

9. Implementation and Delivery Plan

9.1 Recommended Build Order

1. Seed the database with categories and products.
2. Retrieve and display database products in React.
3. Implement member registration, login, session restoration, and logout.
4. Implement the cart in browser storage or the approved persistence layer.
5. Create orders through the backend and persist order items.
6. Support guest checkout and secure guest confirmation.
7. Apply the backend-calculated member discount if included.
8. Add interface language selection.
9. Deploy the client, API, administration interface, and database.
10. Add optional product administration and member order history only if time remains.

Delivery principle: Build one complete vertical path before polishing secondary screens: database product → React catalogue → cart → checkout API → persisted order → confirmation.

9.2 Five-Day Plan

Phase	Work	End-of-day target
Day 1 — Foundation	Confirm scope, stories, wireframes, ER diagram, endpoints, repositories, app scaffolds, Neon database, tables, and seed products.	React and API run locally; products can be retrieved from PostgreSQL.
Day 2 — Backend	Implement authentication, hashing, JWT handling, product/category endpoints, order creation, server-side calculations, endpoint tests, and API documentation.	The shopping process can be demonstrated through API calls.
Day 3 — Frontend	Build layout, routing, homepage, authentication screens, products, search, categories, details, cart, language selector, and API integration.	Users can register, log in, browse products, and manage a cart.
Day 4 — Checkout and deployment	Implement guest/member checkout, discounts, confirmation, protected dashboard, responsive layouts, deployment, and CORS fixes.	The complete flow works in the deployed application.
Day 5 — Validation and submission	Execute test cases, record defects, test languages and checkout roles, verify persistence, complete documentation, prepare presentation, and collect evidence.	The solution and supporting submission materials are demonstration-ready.

9.3 Team Responsibilities

The project is delivered by Weijie, Shukla, and Jade. Final responsibility assignments should reflect the team’s actual contributions; the following allocation is a practical working model.

Workstream	Responsibility
Frontend	React components, routing, responsive layouts, shop experience, cart, authentication screens, and interface integration.
Backend and database	Express endpoints, validation, authentication, PostgreSQL queries, order transactions, server calculations, and Neon integration.
Quality and delivery	Test execution, bug tracking, deployment support, documentation, screenshot evidence, final report, and presentation preparation.
Shared	Scope decisions, code review, integration testing, demonstration rehearsal, and final submission review.

10. Testing and Quality Assurance

10.1 Required Test Coverage

Area	Representative checks
Authentication	Valid and invalid registration, duplicate email, login, refresh persistence, logout, and protected-route access
Products	Retrieval, active filtering, details, search by name/SKU, category filters, loading, empty, and failure states
Cart	Add, duplicate add, quantity updates, removal, totals, persistence, and empty-cart prevention
Checkout	Guest and member flows, validation, backend recalculation, discount, duplicate submission prevention, and database transaction behavior
Orders	Order and item persistence, confirmation retrieval, authorization, guest token access, and optional member history
Internationalization	English/Chinese switching, persistence, fallback strings, and translated validation/interface content
Responsive/accessibility	Desktop/mobile layouts, keyboard navigation, field labels, alternative text, contrast, and no horizontal overflow
Deployment	Production API connectivity, environment variables, CORS, database access, and direct-route refresh behavior

10.2 Evidence to Retain

- Homepage and product catalogue.
- Product details.
- Cart with calculated totals.
- Guest and member checkout.
- Order confirmation and tracking.
- Protected member dashboard and optional order history.

- Vercel deployment status for client, admin, and backend.
- Neon database records showing products, orders, and order items.
- Test-case results and the bug log.

10.3 Demonstration Flow

1. Open the deployed storefront and explain the project purpose.
2. Browse products and categories, then search for an item.
3. Open product details and add an item to the cart.
4. Update the cart and proceed to checkout.
5. Complete a guest or member order and explain server-side pricing.
6. Display confirmation or tracking data retrieved from the stored order.
7. Show the related database records or API response as evidence.

11. Repositories and Deployment

11.1 GitHub Repositories

- Client: <https://github.com/Jade-pang-coder/capstone-project-groupone.git>
- Backend: <https://github.com/shuklasubodh/capstone-project-backend>
- Administration: https://github.com/susutae/project_capstone_admin_react.git

11.2 Deployed Applications

- Backend API: <https://capstone-project-backend-delta.vercel.app/>
- Administration interface: <https://project-capstone-admin-react.vercel.app/#/>
- Client storefront: <https://capstone-project-groupone.vercel.app/>

11.3 Deployed Resource Endpoints

Resource	Endpoint
Users	https://capstone-project-backend-delta.vercel.app/api/users
Categories	https://capstone-project-backend-delta.vercel.app/api/categories
Products	https://capstone-project-backend-delta.vercel.app/api/products
Carts	https://capstone-project-backend-delta.vercel.app/api/carts
Cart items	https://capstone-project-backend-delta.vercel.app/api/cart-items
Orders	https://capstone-project-backend-delta.vercel.app/api/orders
Order items	https://capstone-project-backend-delta.vercel.app/api/order-items

Item-specific routes append `/:id` where supported. Order-item retrieval may also use `/api/order-items/order/:order_id`.

12. Final Completion and Submission Plan

12.1 README and Deployment Guide

The README and deployment guide should enable another developer or assessor to understand, configure, run, and deploy the project.

- Project introduction, purpose, features, and technology stack.
- Installation requirements and local startup instructions.
- Frontend, backend, and database configuration.
- Required environment variables without exposing secret values.
- Vercel and Neon deployment steps.
- Production URLs and API base path.
- Common troubleshooting guidance, including CORS and environment issues.

12.2 Final Report

- Introduction and project objectives.
- System analysis, user journeys, architecture, frontend/backend structure, and database model.
- Implementation of product browsing, cart, checkout, order tracking, API integration, and persistence.
- Test approach, results, defects, and corrective actions.
- Challenges, technical decisions, achievements, limitations, and future improvements.
- Deployment evidence and conclusions.

12.3 Ten-Minute Presentation

Time	Section	Content
1 min	Introduction	Project background, problem, and purpose
1 min	Technology	React, Vite, Express, Vercel, and Neon
3 min	Feature demonstration	Products, details, cart, checkout, and tracking
2 min	Technical implementation	Frontend structure, API integration, and database

3 min	Testing and conclusion	Results, achievements, limitations, and future work
-------	------------------------	---

12.4 Submission Checklist

Deliverable	Status
README completed	Pending
Deployment guide completed	Pending
Final report completed	Pending
Presentation slides prepared	Pending
Ten-minute demonstration rehearsed	Pending
Screenshots collected and organized	Pending
Test results and bug log finalized	Pending
Production URLs and database persistence verified	Pending
Final team review completed	Pending

Appendix A — Initial Test Data

The following seed statements are retained from the original project notebook as implementation reference. Review the table and column names against the final schema before execution. The sample password hashes and contact details are demonstration data only.

1. Insert into categories (Generates IDs 1 to 5)

```
INSERT INTO categories (name, slug, created_at) VALUES ('Living Room', 'living-room',
NOW()), ('Clothing', 'clothing', NOW()), ('Ladies Bags', 'ladies-bags', NOW()), ('Dining &
Kitchen', 'dining-kitchen', NOW()), ('Home Accessories', 'home-accessories', NOW());
```

-- 2. Insert into users (Generates IDs 1 to 5)

```
INSERT INTO users (full_name, email, password_hash, membership_tier,
discount_percentage, created_at, updated_at) VALUES ('Alice Smith',
'alice@example.com',
'$2a$12$elmiTXuWVxfM37uY4JANjO5yA0M7/dZ06Y0A/jZ8dY3a0O.X12345', 'Gold',
10.00, NOW(), NOW()), ('Bob Jones', 'bob@example.com',
'$2a$12$elmiTXuWVxfM37uY4JANjO5yA0M7/dZ06Y0A/jZ8dY3a0O.X12346', 'Silver',
5.00, NOW(), NOW()), ('Charlie Brown', 'charlie@example.com',
'$2a$12$elmiTXuWVxfM37uY4JANjO5yA0M7/dZ06Y0A/jZ8dY3a0O.X12347', 'Bronze',
0.00, NOW(), NOW()), ('Diana Prince', 'diana@example.com',
'$2a$12$elmiTXuWVxfM37uY4JANjO5yA0M7/dZ06Y0A/jZ8dY3a0O.X12348', 'Platinum',
15.00, NOW(), NOW()), ('Evan Wright', 'evan@example.com',
'$2a$12$elmiTXuWVxfM37uY4JANjO5yA0M7/dZ06Y0A/jZ8dY3a0O.X12349', 'Regular',
0.00, NOW(), NOW());
```

-- 3. Insert into products (Generates IDs 1 to 5; maps category_id to 1..5)

```

INSERT INTO products (category_id, sku, title, description, unit_price, image_url,
is_active, created_at, updated_at) VALUES (1, 'SOFA-001', 'Modern 3-Seater Sofa',
'Comfortable fabric sofa for living room', 499.99, 'https://example.com/images/sofa.jpg',
true, NOW(), NOW()), (2, 'CLT-002', 'Cotton Casual Shirt', '100% organic cotton shirt',
29.99, 'https://example.com/images/shirt.jpg', true, NOW(), NOW()), (3, 'BAG-003', 'Leather
Tote Bag', 'Spacious genuine leather handbag', 89.50,
'https://example.com/images/bag.jpg', true, NOW(), NOW()), (4, 'TRY-004', 'Wooden Dining
Tray Set', 'Set of 3 handcrafted serving trays', 45.00, 'https://example.com/images/tray.jpg',
true, NOW(), NOW()), (5, 'ACC-005', 'Ceramic Table Lamp', 'Warm lighting decorative
lamp', 34.99, 'https://example.com/images/lamp.jpg', true, NOW(), NOW());

-- 4. Insert into carts (Generates IDs 1 to 5; maps user_id to 1..4, null)
INSERT INTO carts (user_id, session_token, created_at, updated_at) VALUES (1,
'sess_abc123', NOW(), NOW()), (2, 'sess_def456', NOW(), NOW()), (3, 'sess_ghi789',
NOW(), NOW()), (4, 'sess_jkl012', NOW(), NOW()), (NULL, 'sess_guest999', NOW(),
NOW());

-- 5. Insert into cart_items (Generates IDs 1 to 5; maps cart_id and product_id to 1..5)
INSERT INTO cart_items (cart_id, product_id, quantity, created_at, updated_at) VALUES
(1, 1, 1, NOW(), NOW()), (1, 4, 2, NOW(), NOW()), (2, 2, 3, NOW(), NOW()), (3, 3, 1,
NOW(), NOW()), (5, 5, 1, NOW(), NOW());

-- 6. Insert into orders (Generates IDs 1 to 5; maps user_id to 1..4, null)
INSERT INTO orders (user_id, order_code, auth_token, customer_type, customer_name,
customer_email, customer_phone, shipping_address, subtotal_amount, discount_amount,
total_amount, status, created_at) VALUES (1, 'ORD-2026-001', 'auth_token_1',
'Registered', 'Alice Smith', 'alice@example.com', '+1234567890', '123 Main St, New York,
NY', 589.99, 58.99, 531.00, 'completed', NOW()), (2, 'ORD-2026-002', 'auth_token_2',
'Registered', 'Bob Jones', 'bob@example.com', '+1234567891', '456 Oak Ave, Brooklyn,
NY', 89.97, 4.50, 85.47, 'processing', NOW()), (3, 'ORD-2026-003', 'auth_token_3',
'Registered', 'Charlie Brown', 'charlie@example.com', '+1234567892', '789 Pine Rd,
Queens, NY', 89.50, 0.00, 89.50, 'pending', NOW()), (4, 'ORD-2026-004', 'auth_token_4',
'Registered', 'Diana Prince', 'diana@example.com', '+1234567893', '321 Elm St,
Manhattan, NY', 499.99, 75.00, 424.99, 'shipped', NOW()), (NULL, 'ORD-2026-005',
'auth_token_5', 'Guest', 'Eve Guest', 'eve@example.com', '+1234567894', '654 Maple Dr,
Staten Island, NY', 45.00, 0.00, 45.00, 'completed', NOW());

-- 7. Insert into order_items (Generates IDs 1 to 5; maps order_id and product_id to 1..5)
INSERT INTO order_items (order_id, product_id, sku, product_title, unit_price, quantity,
subtotal) VALUES (1, 1, 'SOFA-001', 'Modern 3-Seater Sofa', 499.99, 1, 499.99), (1, 4,
'TRY-004', 'Wooden Dining Tray Set', 45.00, 2, 90.00), (2, 2, 'CLT-002', 'Cotton Casual
Shirt', 29.99, 3, 89.97), (3, 3, 'BAG-003', 'Leather Tote Bag', 89.50, 1, 89.50), (4, 1,
'SOFA-001', 'Modern 3-Seater Sofa', 499.99, 1, 499.99);

```

Appendix B — Project Decisions at a Glance

Decision	Agreed direction
Guest access	Guests browse and check out without authentication.
Payment	No online payment gateway in the MVP.
Cart persistence	Use localStorage for the simplest MVP; database cart tables are optional.
Pricing authority	The backend retrieves current product prices and calculates final totals.
Member benefit	Use one configurable 10% discount if the scope extension is included.
Language	Translate the primary interface into English and Chinese; catalogue translation is optional.
Historical accuracy	Store SKU, product name, and unit price snapshots in order_items.
Deployment	Use Vercel for applications and Neon for PostgreSQL.
Completion priority	A stable product-to-order vertical flow takes precedence over optional features.